

13C项目之BookLessor

一、需求分析

几乎所有学校都有图书馆，一方面给学生查阅知识，另一方面可以提供借书服务，本项目旨在为各个学校提供一个支持租用业务的图书管理软件。

显然这样一个软件主要处理三类数据：

1. 用户信息：用户名、密码、类型、联系方式（字符串）、证件号码(学生证编号或身份证号码字符串)、证件姓名、出生日期
2. 图书信息：书名、册数、作者、版号、ISBN(长度为13、纯数字字符的字符串)、出版日期、库存数量、原价、租用价、租用次数
3. 租用订单信息：租用日期、ISBN、用户名、订单状态（关闭和未关闭）、归还日期、处理方式（正常归还、损毁赔偿）

用户可以分为三类：

1. 学生（租用价格八折，限制未关闭订单数量为3）

支持的操作：

1. 图书查询：按书名查询、按热度从高到低查询
2. 订单查询：历史已完成订单查询、未关闭订单查询
3. 修改密码

2. 老师（租用价格九折，限制未关闭订单数量为5）

支持的操作：

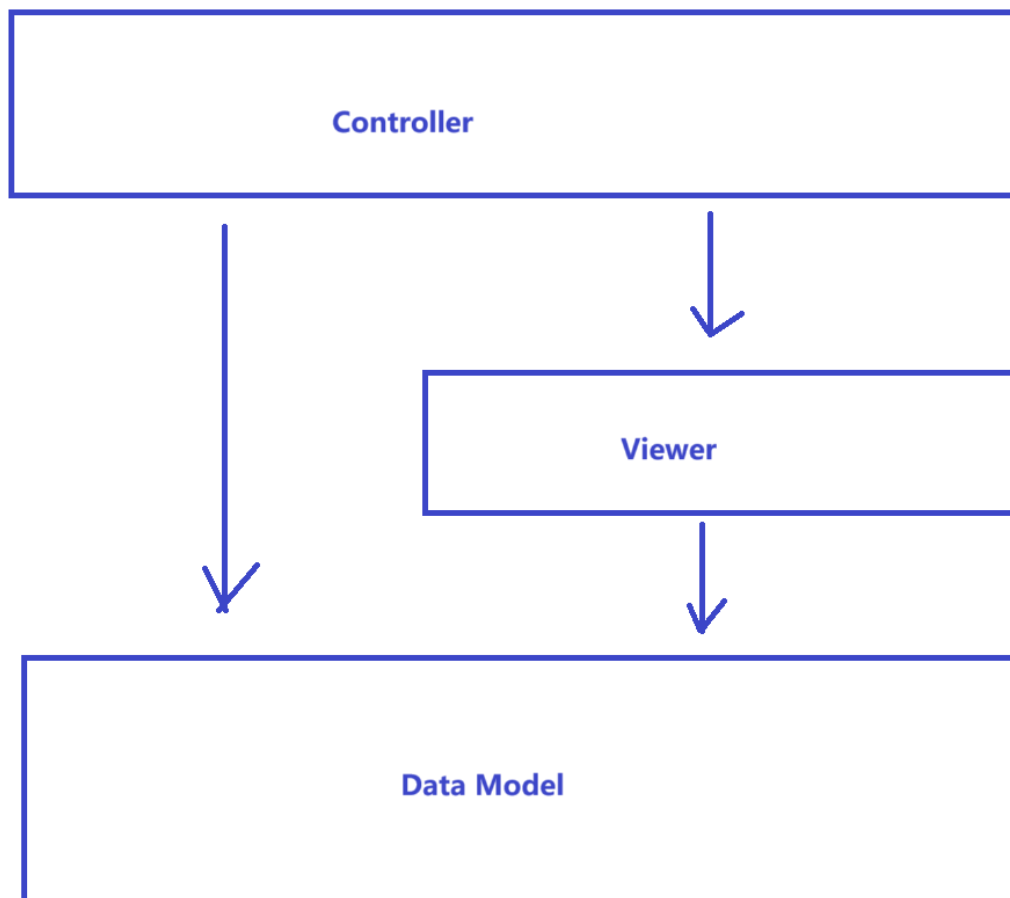
1. 图书查询：按书名查询、按热度从高到低查询
2. 订单查询：历史已完成订单查询、未关闭订单查询
3. 修改密码

3. 管理员（无租用权限）

支持的操作：

1. 新增用户
2. 图书入库
3. 新增订单
4. 关闭订单
5. 销售额查询（以归还日期为准）
6. 热度查询
7. 修改密码

二、概要设计



项目整体划分为三大模块：

1. 数据模型：DataModel

负责核心数据的组织管理：内存数据结构、外存文件格式以及相应的各种操作函数

进一步分为三个子模块：

1. 用户信息

方案1：一个链表（可单向、可双向、非循环、带头节点）、文件格式可以采用文本文件或非文本文件

方案2：hash链表分别管理学生用户、老师用户和管理员用户数据、三种用户数据分别用三个文件（student.usr teacher.usr admin.usr）存放，三个文件可以采用文本文件或非文本文件

2. 图书信息

一个链表（可单向、可双向、非循环、带头节点），外存文件（book.dat）格式可以采用文本文件或非文本文件

3. 订单信息

方案1：一个链表（可单向、可双向、非循环、带头节点）、文件格式可以采用文本文件或非文本文件

方案2：hash链表分别管理未关闭订单、已关闭订单信息，两种订单数据分别用两个文件（closed.dat opened.dat）存放，两个文件可以采用文本文件或非文本文件

2. 界面：Viewer

进一步划分为以下几个子模块：

1. 主界面
2. 学生登录后的界面
3. 老师登录后的界面
4. 管理员登录后的界面
5. 公共界面

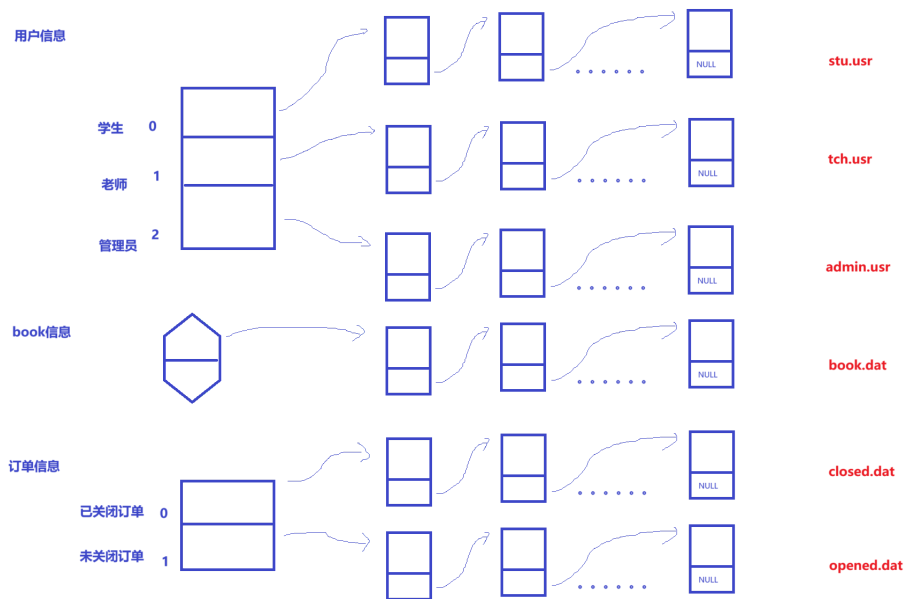
3. 控制器：Controller

进一步划分为以下几个子模块：

1. main
2. 登录功能
3. 学生登录后各个功能的实现
4. 老师登录后各个功能的实现
5. 管理员登录后各个功能的实现

三、详细设计

3.1 各模块接口设计



用户类型、订单状态、订单处理方式设计枚举

联系方式、证件号码、ISBN均采用字符串，代码中需要验证其是否合法

```
//book_lesson.h
#ifndef BOOK_LESSOR_H
#define BOOK_LESSOR_H

#include "book_model.h"
#include "order_model.h"
#include "user_model.h"
#include "viewer.h"

#endif
```

```
//bl_public.h
#ifndef BOOK_LESSOR_PUBLIC_H
#define BOOK_LESSOR_PUBLIC_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ADMIN_NAME "admin"
```

```

#define ADMIN_PASS "123456"

#define ISBN_LEN 13
#define BOOK_NAME_LEN 40
#define USER_NAME_LEN 20
#define PASS_LEN 20
#define PHONE_LEN 14
#define ID_CARD_LEN 18
#define STU_CARD_LEN 16

// +3 for Byte Alignment
#define ISBN_BUF_LEN (ISBN_LEN + 3)

// +2 for Byte Alignment
#define PHONE_BUF_LEN (PHONE_LEN + 2)

#define CARD_BUF_LEN 20

#define BOOK_DAT_FILE "./book.dat"

#define USER_STU_FILE "./student usr"
#define USER_TCH_FILE "./teacher usr"
#define USER_ADMIN_FILE "./admin usr"

#define CLOSED_ORDER_FILE "./closed.dat"
#define OPENED_ORDER_FILE "./opened.dat"

struct date
{
    int y;
    int m;
    int d;
};

enum infotype
{
    DT_USER = 1,
    DT_BOOK,
    DT_ORDER
};

struct node
{
    void *pdata;
    int infotype;//enum infotype

```

```

    struct node *pNext;
};

//无头节点单向链表操作函数
int get_len_of_linklist(struct node *phead);
int destroy_node(struct node *pnode);
int insert_node_at_tail(struct node **phead, struct node *pnew);
int insert_node_at_head(struct node **phead, struct node *pnew);
int remove_node(struct node **phead, struct node *pnode);
int clear_linklist(struct node **phead);

#endif

```

```

//book_model.h
#ifndef BOOK_MODEL_H
#define BOOK_MODEL_H

#include "bl_public.h"

//书名、册数、作者、版号、ISBN(长度为13、纯数字字符的字符串)、出版日期、库存数量、原价、租用
价、租用次数
struct bookinfo
{
    char isbn[ISBN_BUF_LEN];
    char name[BOOK_NAME_LEN];
    int volume;
    int version;
    struct date pubdate;
    int stock;
    float sprice;
    float lprice;
    int lcnt;
};

struct bookslinklist
{
    struct node *pfirst;

    int cnt;
};

struct node *create_book_node(struct bookinfo *pdata);
int read_book_from_file(struct bookslinklist *pbsll, char *filename);
int save_book_to_file(struct bookslinklist *pbsll, char *filename);
struct node **sort_books_by_lcnt(struct bookslinklist *pbsll);
struct node *search_book_by_isbn(struct bookslinklist *pbsll, char *pisbn);

```

```

int search_book_by_name(struct bookslinklist *pbsll, char *pname, struct node
**ppout, int maxcnt); //返回实际找到的项数, maxcnt期望最大的项数

#endif

```

```

//user_model.h
#ifndef USER_MODEL_H
#define USER_MODEL_H

#include "bl_public.h"

enum usertype
{
    UT_STU,
    UT_TCH,
    UT_ADMIN
};

//用户名、密码、类型、联系方式（字符串）、证件号码（学生证编号或身份证号码字符串）、证件姓名、出生日期
struct userinfo
{
    char username[USER_NAME_LEN];
    char password[PASS_LEN];
    int type;
    char phone[PHONE_BUF_LEN];
    char paper[CARD_BUF_LEN];
    char cardname[USER_NAME_LEN];
    struct date birth;
};

#define UserHash(t) ((t)%3)

struct node *create_user_node(struct userinfo *pdata);
int read_user_from_file(struct node **pphead, char *filename);
int save_user_to_file(struct node *phead, char *filename);

int read_all_user(struct node **pphasharr, int cnt);
int save_all_user(struct node **pphasharr, int cnt);

struct node *search_user_by_name(struct node **pphasharr, int ut, char *name);

#endif

```

```

//order_model.h
#ifndef ORDER_MODEL_H
#define ORDER_MODEL_H

#include "bl_public.h"

enum ORDER_STATUS
{
    CLOSED = 0,
    OPENED,
};

enum HANDLER_WAY
{
    UNHANDLED,
    NORMAL,
    REDRESS,
};

//租用日期、ISBN、用户名、订单状态（关闭和未关闭）、归还日期、处理方式（正常归还、损毁赔偿）
struct orderinfo
{
    struct date ldate;
    char isbn[ISBN_BUF_LEN];
    char username[USER_NAME_LEN];
    int status;
    struct date rdate;
    int hway;
    float lastprice;
};

#define OrderHash(s) ((s)%2)

struct node *create_order_node(struct orderinfo *pdata);
int read_order_from_file(struct node **pphead, char *filename);
int save_order_to_file(struct node *phead, char *filename);

int read_all_order(struct node **pphasharr, int cnt);
int save_all_order(struct node **pphasharr, int cnt);

struct node **search_opened_order_by_name(struct node **pphasharr, char
*username, int *pcnt);
struct node **search_closed_order_by_name(struct node **pphasharr, char
*username, int *pcnt);
struct node *search_node_by_isbn(struct node **ppopened, int cnt, char *pisbn);

```



```
float count_month_sales(struct node **pphasharr,int year,int month);
float count_year_sales(struct node **pphasharr,int year);

#endif
```

//第一次运行时,各种外存文件可能不存在或文件长度为0,此时只有一个默认的用户名为admin的用户,为了判断这种情况,可以调用如下函数:

```
int is_exist_file(char *filename)
```

```
{
    FILE *pf = NULL;

    pf = fopen(filename,"r");
    if(NULL == pf)
    {
        return 0;
    }
    else
    {
        int len = 0;
        fseek(pf,0,SEEK_END);
        len = ftell(pf);
        if(len <= 0)
        {
            fclose(pf);
            pf = NULL;
            return 0;
        }
        else
        {
            fclose(pf);
            pf = NULL;
            return 1;
        }
    }
}
```

//参考调用:

```
if(is_exist_file("????"))
```

```
{
    从文件中读取信息数据到相应链表中
}
else
{
    根据不同文件作用要么什么都不做,要么仅向管理员用户链表中添加默认用户admin对应的节点
}
```

3.2 源码目录设计

```
booklessor/
├── bin      可执行文件存放在该目录下，程序执行在此目录下进行
├── inc
│   ├── book_model.h 图书信息数据结构相关的操作函数声明、类型设计，宏定义等对外接口写在此文件中
│   ├── order_model.h 订单信息数据结构相关的操作函数声明、类型设计，宏定义等对外接口写在此文件中
│   ├── user_model.h 用户信息数据结构相关的操作函数声明、类型设计，宏定义等对外接口写在此文件中
│   └── viewer.h      viewer模块下所有.c对外提供的函数声明写在此文件中
├── src
│   ├── controller
│   │   ├── admin_ctl.c 管理员登录后的界面功能实现
│   │   ├── main.c      主界面功能实现
│   │   ├── stu_ctl.c   学生登录后的界面功能实现
│   │   └── tea_ctl.c   教师登录后的界面功能实现
│   ├── Makefile
│   ├── model
│   │   ├── book_model.c 图书信息数据结构相关操作函数的实现以及读写文件的操作函数实现
│   │   ├── order_model.c 订单信息数据结构相关操作函数的实现以及读写文件的操作函数实现
│   │   └── user_model.c 用户信息数据结构相关操作函数的实现以及读写文件的操作函数实现
│   └── viewer
│       ├── admin_ui.c 管理员登录后的各种界面函数
│       ├── main_ui.c 主界面相关的界面函数
│       ├── public_ui.c 公共的界面函数
│       ├── stu_ui.c 学生登录后的各种界面函数
│       └── tea_ui.c 教师登录后的各种界面函数
```

inc目录下添加两个头文件：

```
├── bl_public.h 为三个model.h文件提供共同信息，三个model.h文件都应该首先包含本文件
├── book_lessor.h 包含三个model.h文件和viewer.h，为.c文件提供#include调用
```

model目录下添加一个.c文件：public_model.c对应bl_public.c中的接口函数实现

```
booklessor/
├── bin
├── inc
│   ├── bl_public.h
│   ├── book_lessor.h
│   ├── book_model.h
│   ├── order_model.h
│   ├── user_model.h
│   └── viewer.h
└── src
    ├── controller
    │   ├── admin_ctl.c
    │   ├── main.c
    │   ├── stu_ctl.c
    │   └── tea_ctl.c
    ├── Makefile
    ├── model
    │   ├── book_model.c
    │   ├── order_model.c
    │   ├── public_model.c
    │   └── user_model.c
    └── viewer
        ├── admin_ui.c
        ├── main_ui.c
        ├── public_ui.c
        ├── stu_ui.c
        └── tea_ui.c
```

stu_ctl.c和tea_ctl.c可以合为unadmin_ctl.c

stu_ui.c和tea_ui.c可以合为unadmin_ui.c

```

//main.c
int main(int argc, char *argv[])
{
    int cmd = 0;
    int exitflag = 0;
    //.....
    while(1)
    {
        cmd = main_ui();
        switch(cmd)
        {
            case 1:
                .....
                break;
            case 2:
                .....
                break;
            case 3:
                ret = admin_login(.....);
                if(!ret)
                {
                    admin_op(.....);
                }
                break;
            case 0:
                exitflag = 1;
                break;
        }
        if(exitflag)
        {
            break;
        }
    }
    //...
}

//main_ui.c
int main_ui()
{
    int op = 0;
    printf("Please select your operator:\n")
    printf("1. Student Login\n");
    printf("2. Teacher Login\n");
    printf("3. Admin Login\n");
    printf("0. Exit\n");
    scanf("%d", &op);
}

```

```

    while(getchar() != '\n')
    {
    }

    return op;
}

//admin_ctl.c
int admin_op(.....)
{
    int cmd = 0;
    int backflag = 0;
    //.....
    while(1)
    {
        cmd = admin_ui();
        switch(cmd)
        {
            case 1:
                .....
                break;
            case 2:
                .....
                break;
            case 3:
                .....
                break;
            .....
            case 0:
                backflag = 1;
                break;
        }
        if(backflag)
        {
            break;
        }
    }
    //...
}

```

四、进度计划

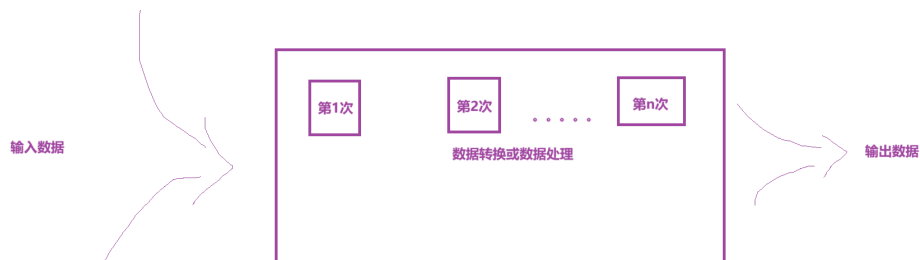
4.1上午 ----- 完成model

4.1下午 + 4.2---15: 00前完成项目

五、备注知识点

5.1 项目设计

从核心数据的组织管理入手 -----内存、外存



影响项目规模的要素:

1. 输入数据来源方式的多寡: 用户输入、文件读入、网络传输、硬件检测
2. 输出数据形式的多寡: 命令行界面输出、图形界面输出、写入文件、网络传输、硬件 (例如打印机) 输出
3. 数据处理过程步骤的多寡: 即转换次数

5.2 MVC模型

MVC:

1. M: Model ---- data model 数据模型
2. V: Viewer ----- 界面: 用户输入数据的界面、向用户展示处理结果的输出界面
3. C: Controller -----控制器, 组合M和V形成程序最终功能